

JTAG Debug Mode — Vitis Unified IDE

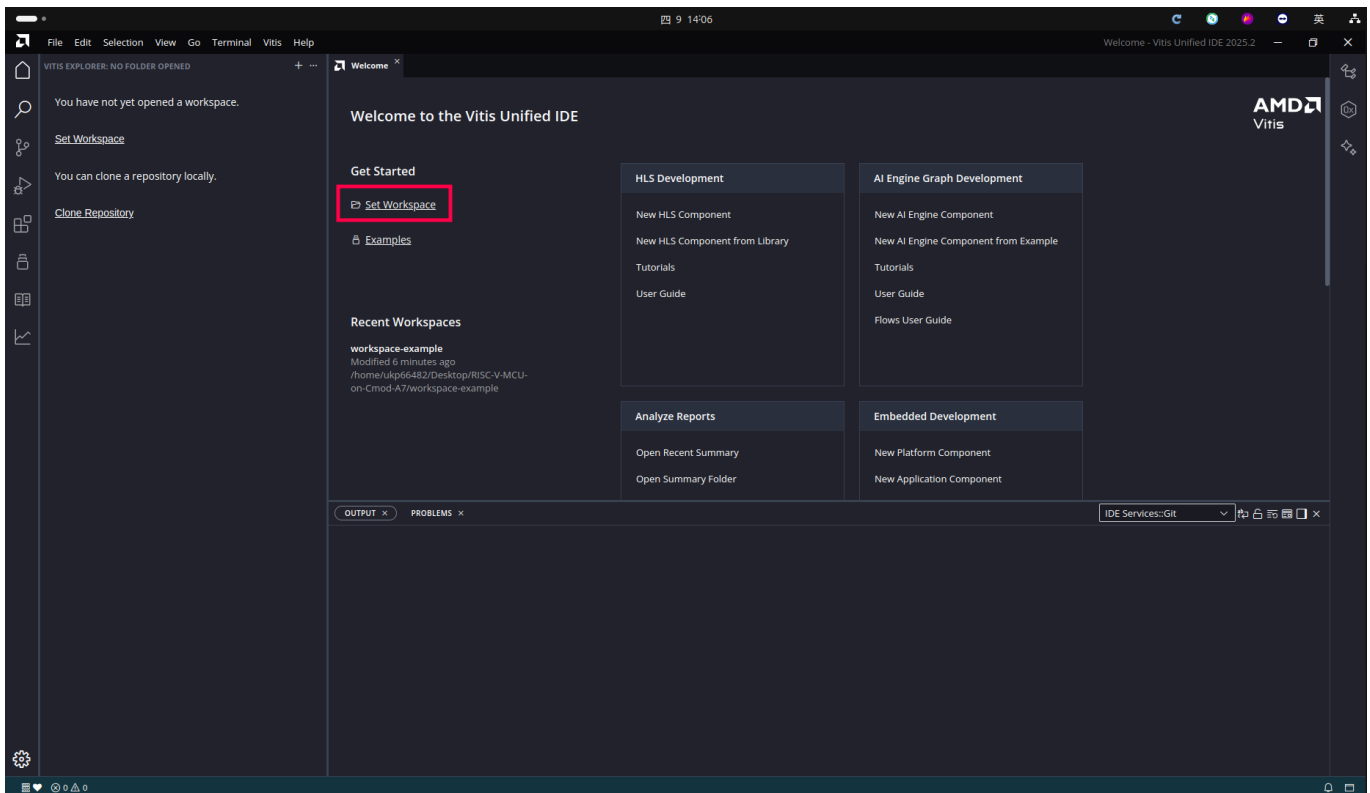
This guide walks through loading and debugging a MicroBlaze RISC-V application over JTAG using the **AMD Vitis Unified IDE**.

Prerequisites

- A hardware design exported as an `.xsa` file from Vivado
- Vitis Unified IDE installed
- Cmod A7-35T board connected to the host via USB

1. Open Vitis Workspace

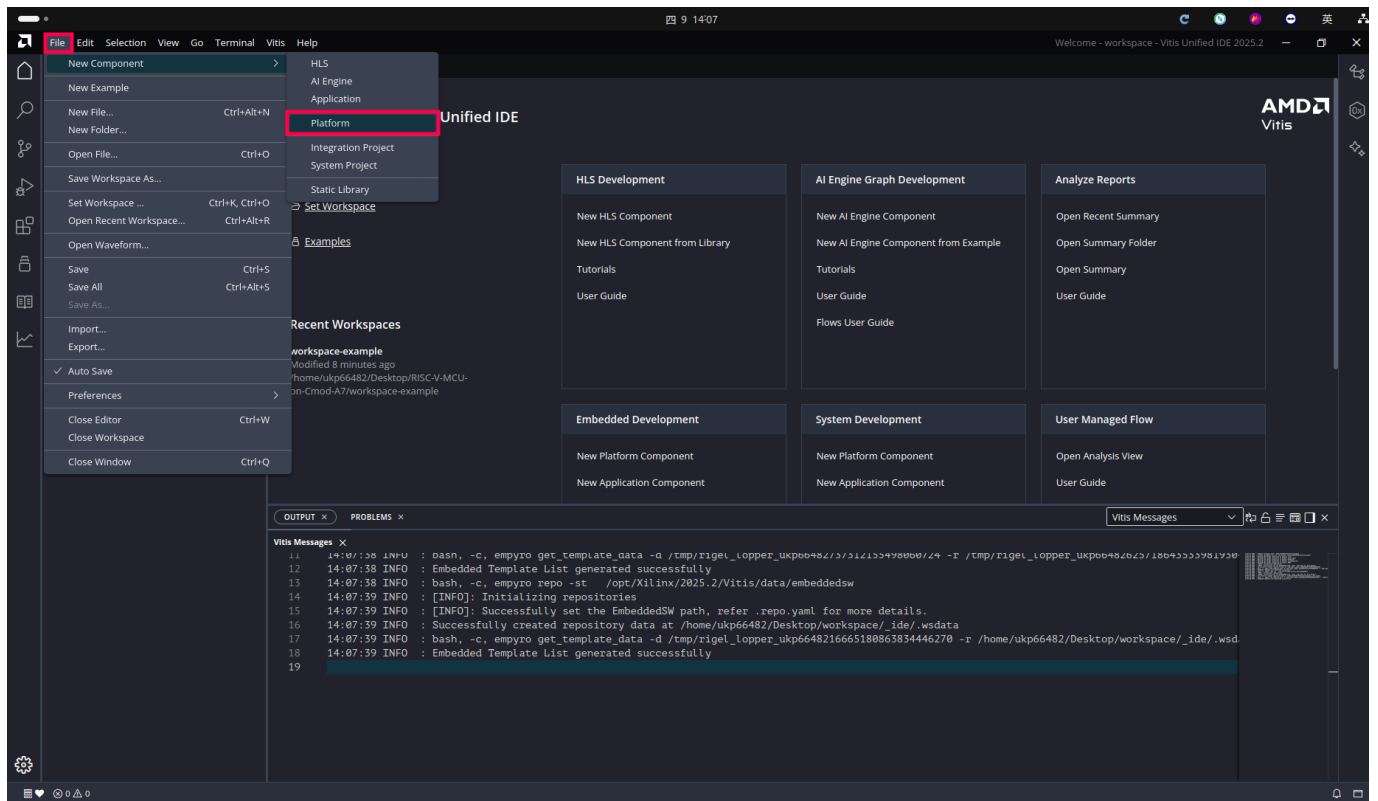
Launch the Vitis Unified IDE. On the Welcome page, click **Open Workspace** to select or create a working directory.



2. Create a Platform

2.1 New Platform Component

From the menu bar, select **File > New > Platform** to create a new platform component.



2.2 Name and Location

On the **Name and Location** page:

- **Component name:** enter `platform`
- **Component location:** choose your workspace directory

Click **Next** to continue.

Create Platform Component ✕

[Name and Location](#) > [Flow](#) > [OS and Processor](#) > [Summary](#)

Name and Location

Choose a name for your component and specify a directory where the component data files will be stored.

Component name

Component location [Browse](#)

Component will be created at /home/ukp66482/Desktop/workspace/platform

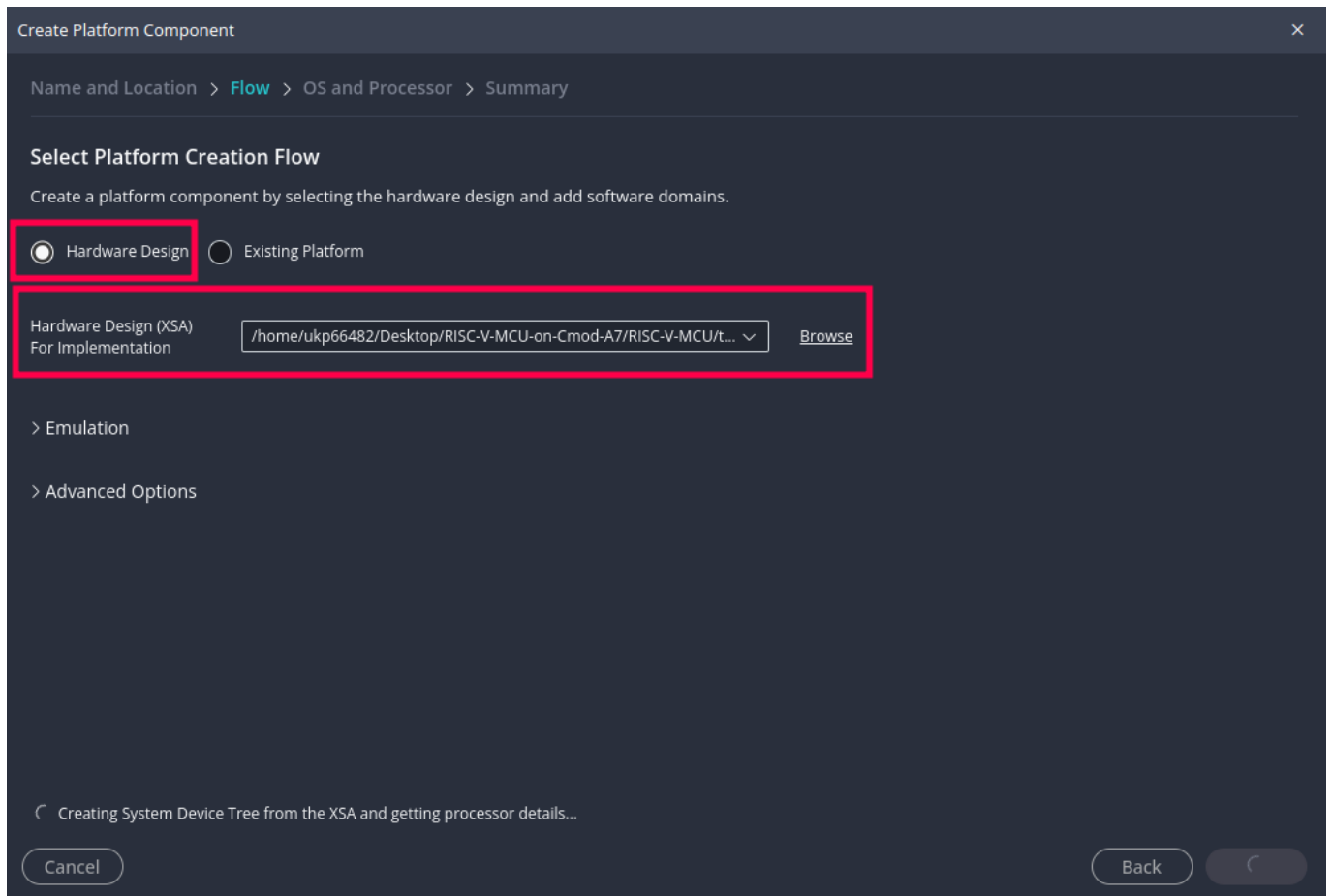
Cancel Next

2.3 Select Hardware Design (XSA)

On the **Flow** page:

- Select **Hardware Design**
- In the **Hardware Design (XSA) For Implementation** field, browse to the `.xsa` file exported from Vivado

Wait for the tool to create the System Device Tree and retrieve processor details.

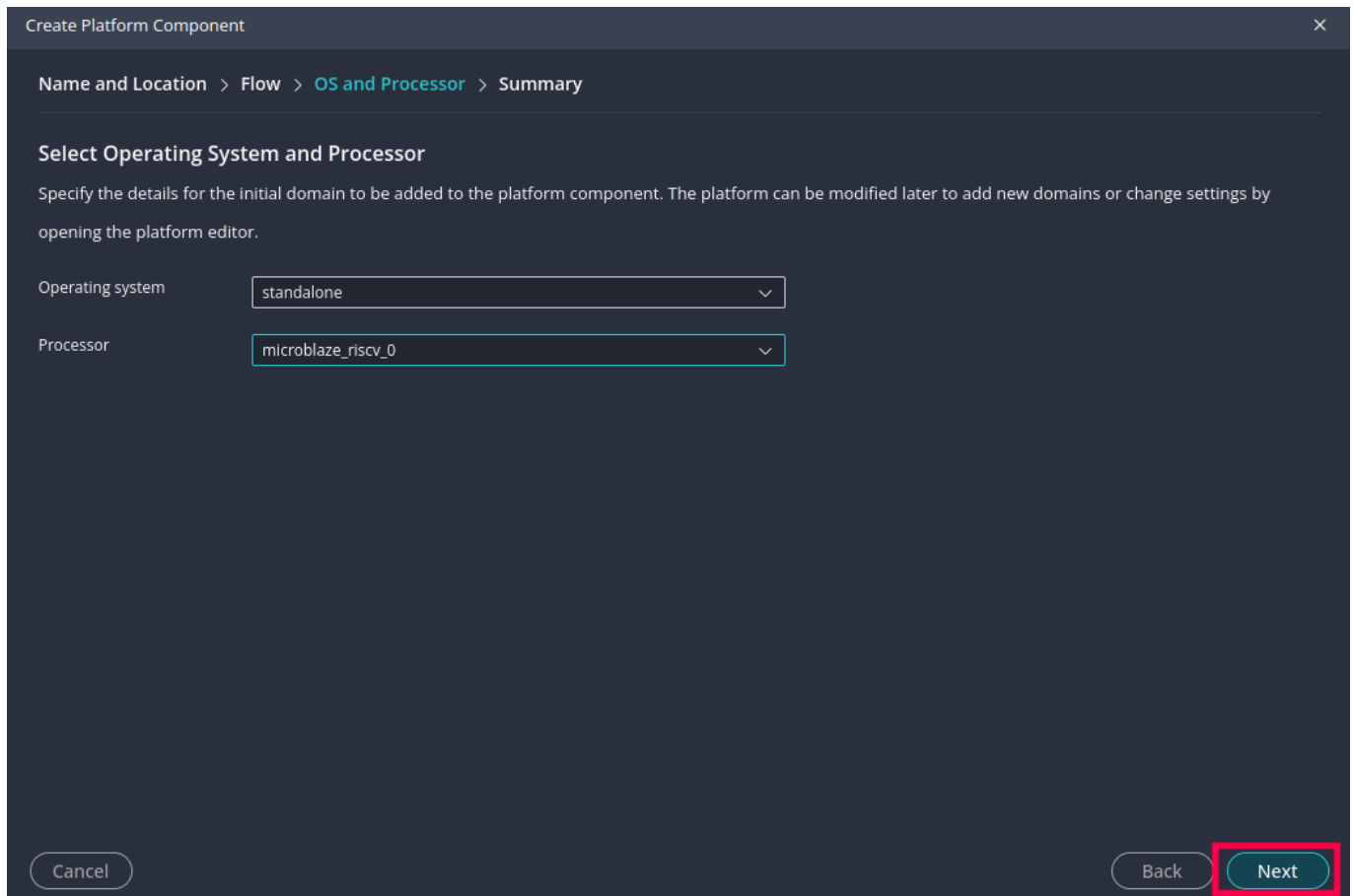


2.4 Select Operating System and Processor

On the **OS and Processor** page:

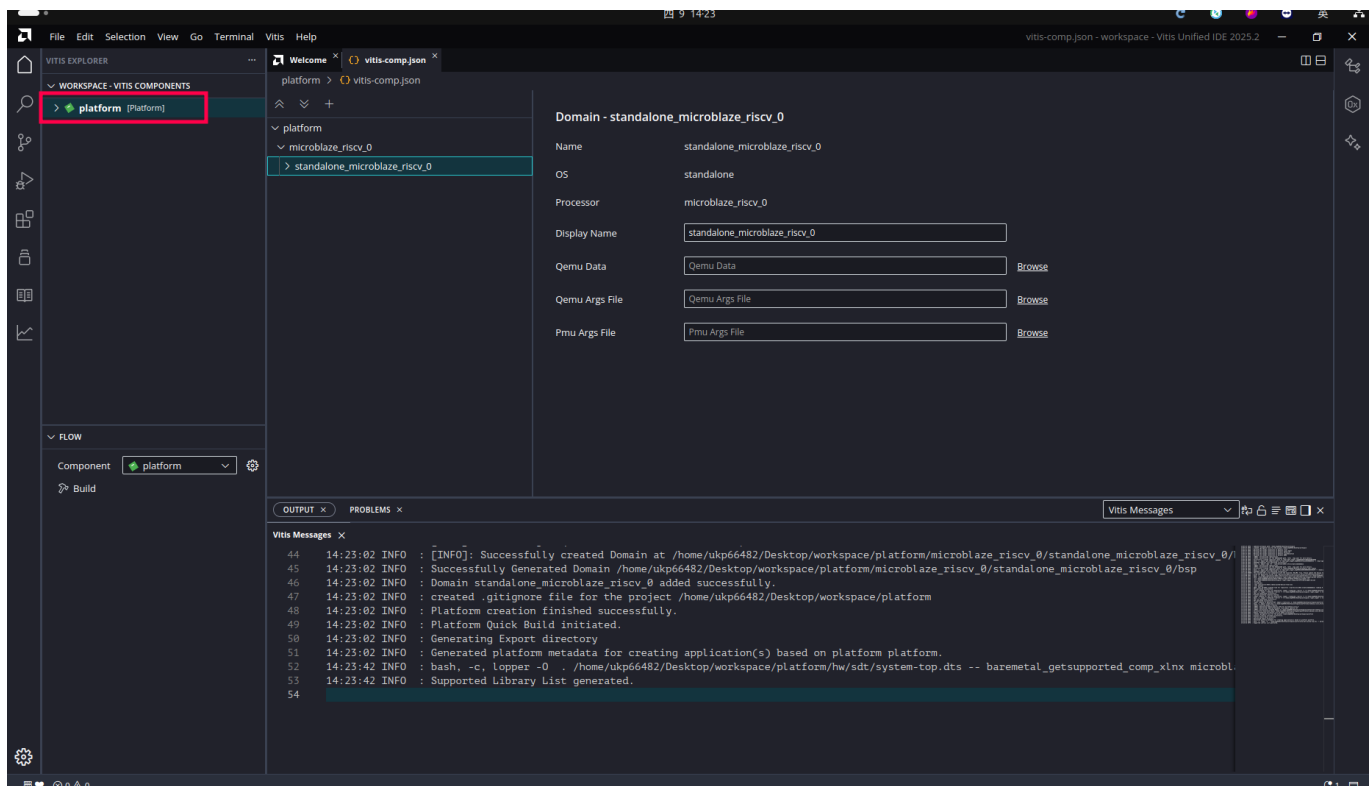
- **Operating system:** standalone
- **Processor:** microblaze_riscv_0

Click **Next** to finish.



2.5 Platform Created

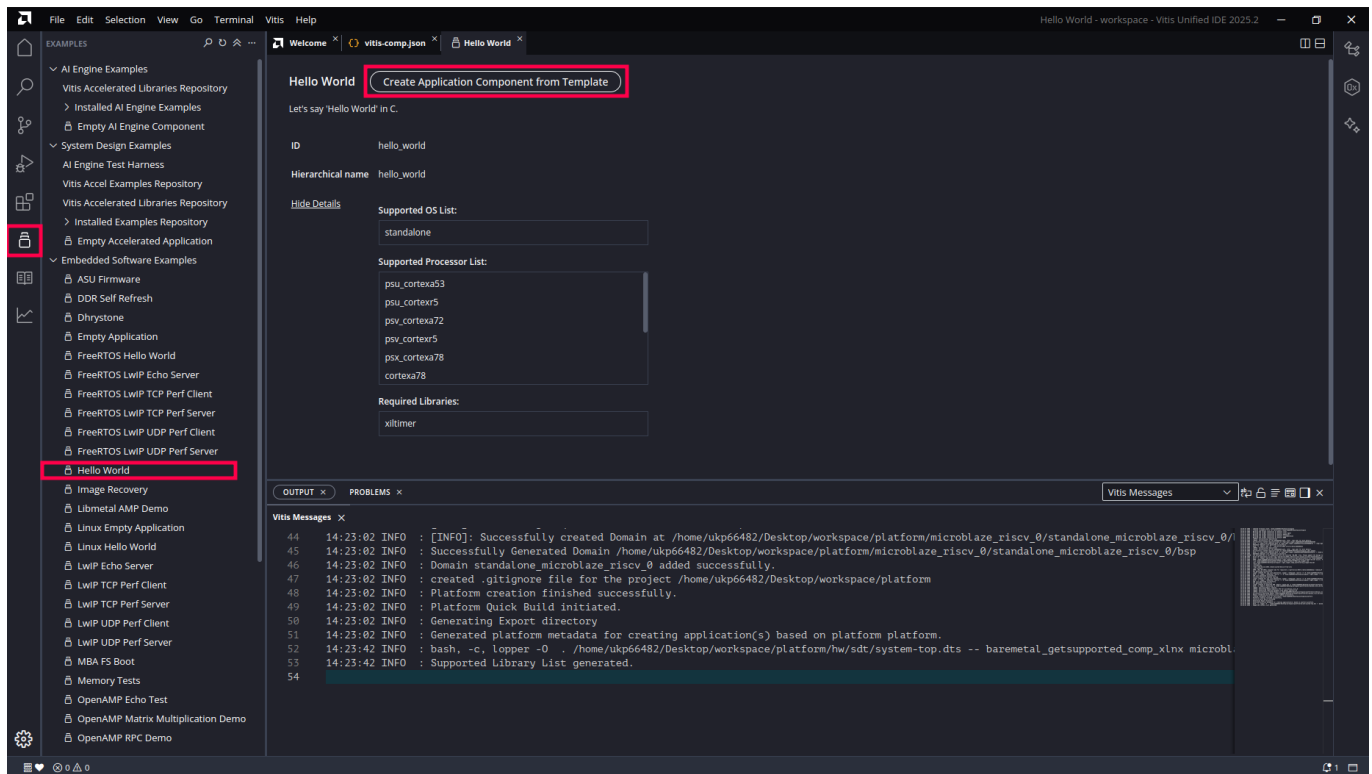
Once created, Vitis displays the platform's Domain settings where you can verify the processor and configuration.



3. Create an Application

3.1 Create from Template

In the left-side template list, select **Hello World**, then click **Create Application Component from Template**.



3.2 Set Application Name

On the **Name and Location** page:

- **Component name:** enter your application name (e.g. `GPIO_test`)

Click **Next** to continue.

Create Application Component - Hello World

[Name and Location](#) > [Hardware](#) > [Domain](#) > [Summary](#)

Name and Location

Choose a name for your component and specify a directory where the component data files will be stored.

Component name

GPIO_test

Component location

/home/ukp66482/Desktop/workspace

Browse

Component will be created at /home/ukp66482/Desktop/workspace/GPIO_test

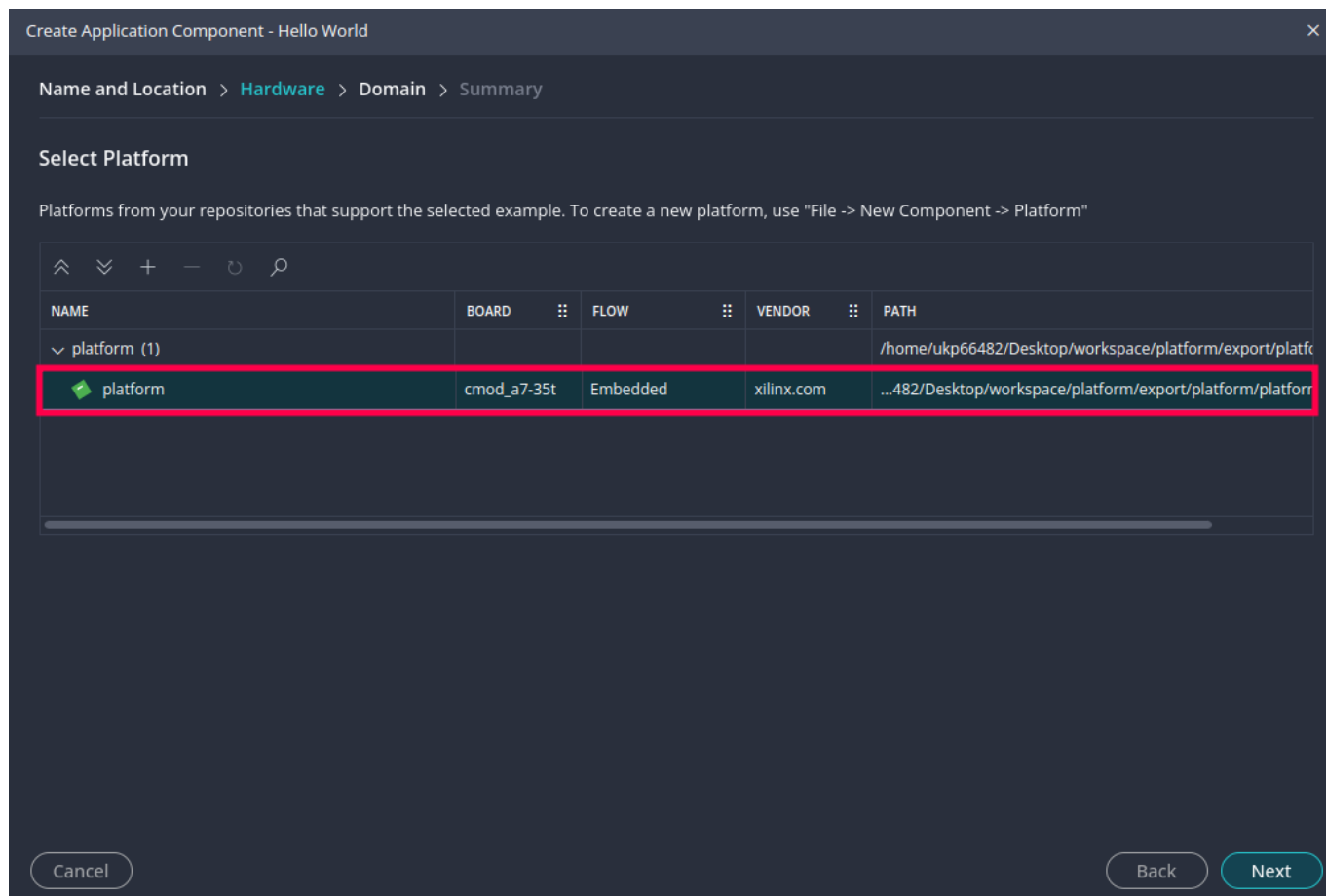
Cancel

Next

3.3 Select Platform

On the **Hardware** page, select the previously created **platform** (Board: `cmmod_a7-35t`).

Click **Next** to continue.

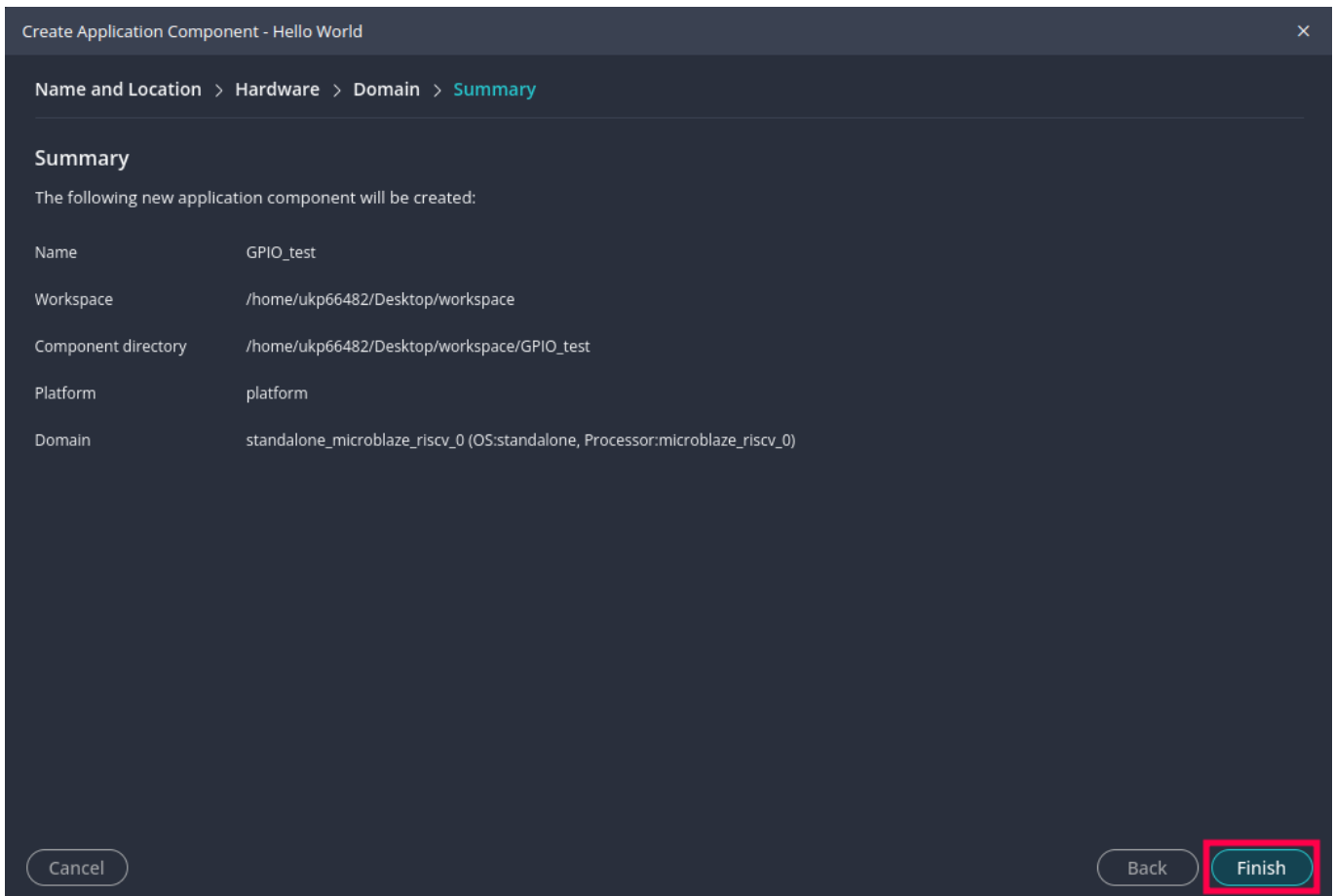


3.4 Review Summary and Finish

Review the application settings:

Field	Value
Name	GPIO_test
Platform	platform
Domain	standalone_microblaze_riscv_0 (OS:standalone, Processor:microblaze_riscv_0)

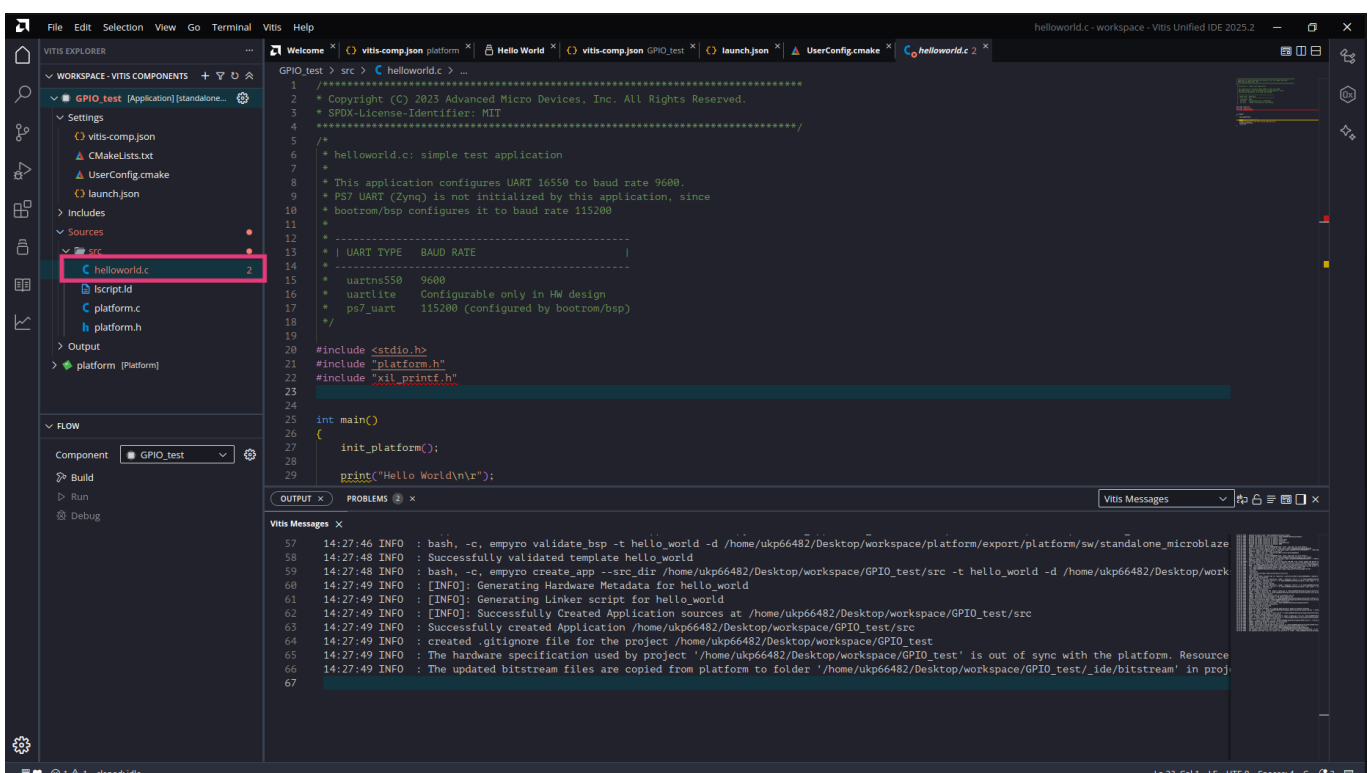
Click **Finish** to create the application.



4. Write Application Code

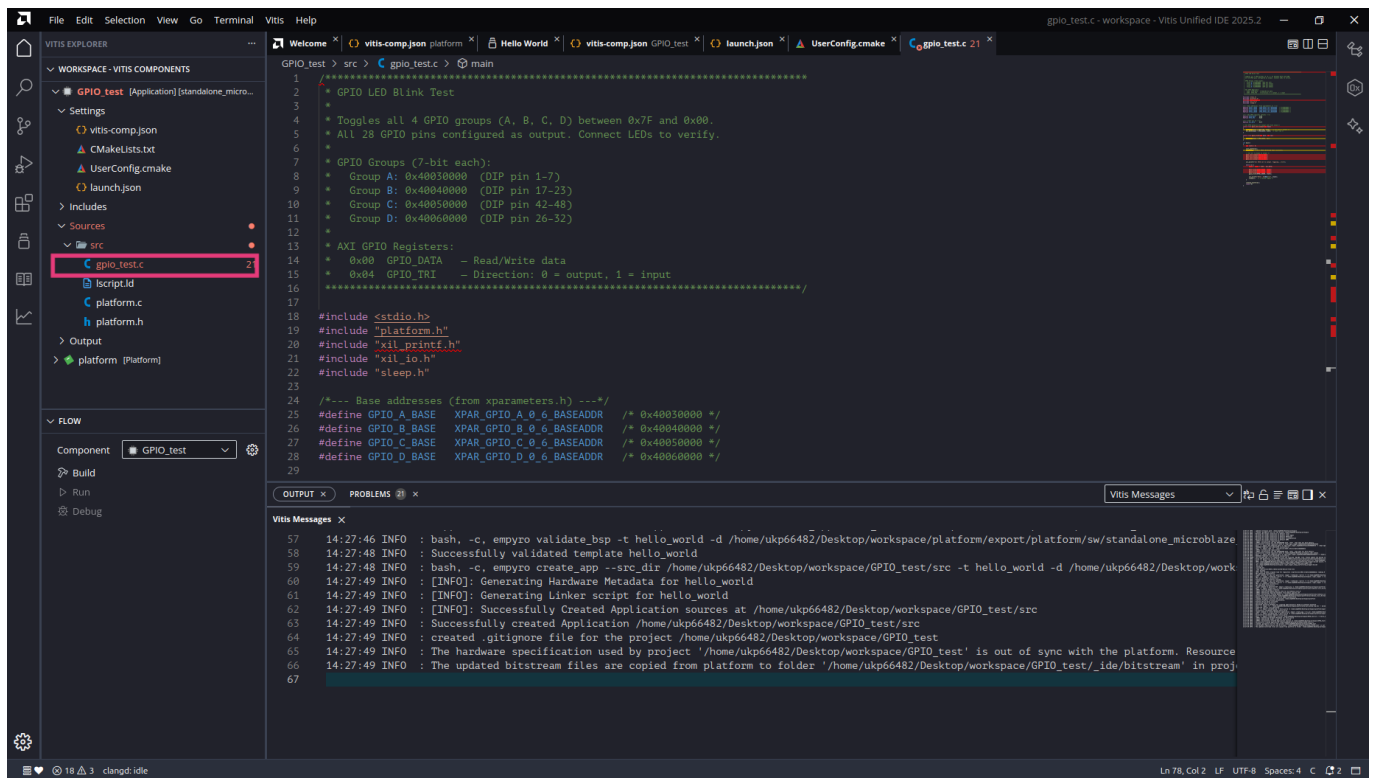
4.1 Default Source Code

After creation, a default `helloworld.c` file is placed in the `src` directory. You can modify this file directly or add your own source files.



4.2 Add Source Files

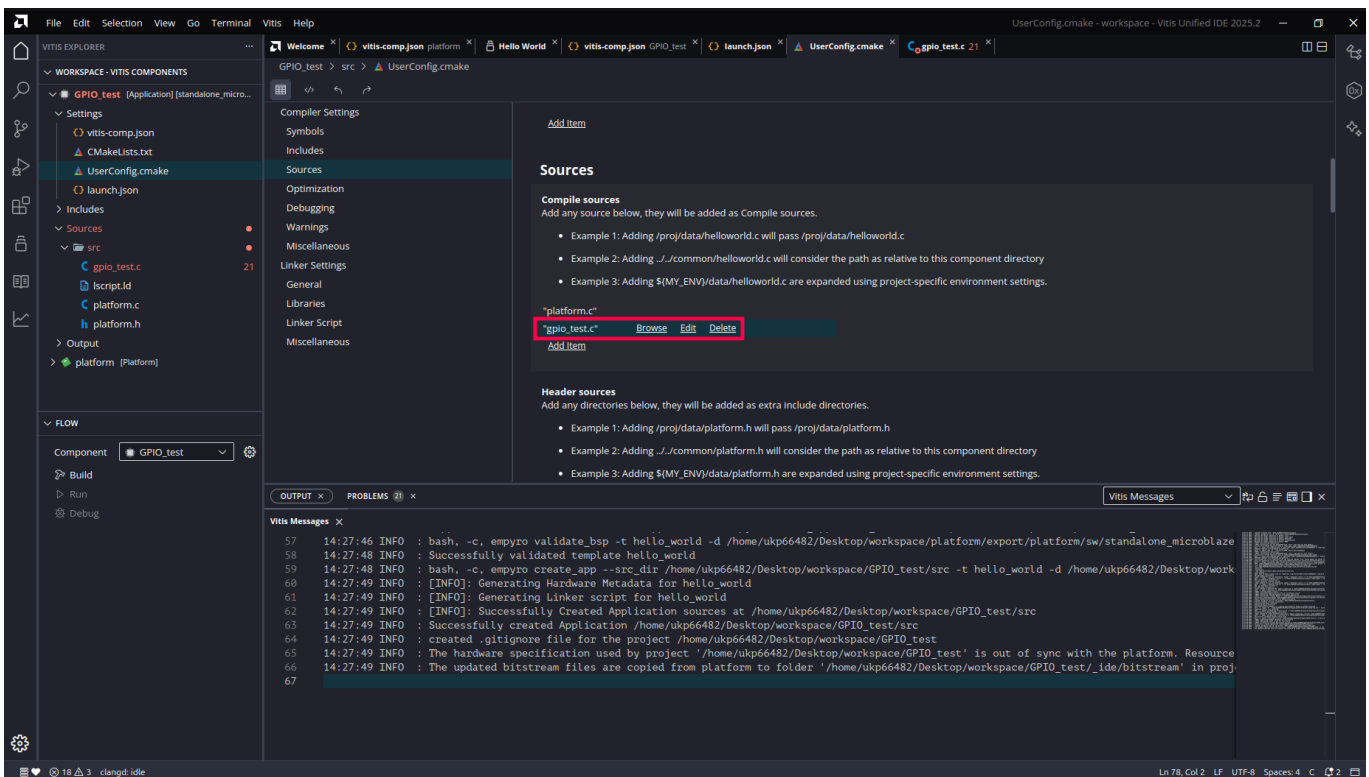
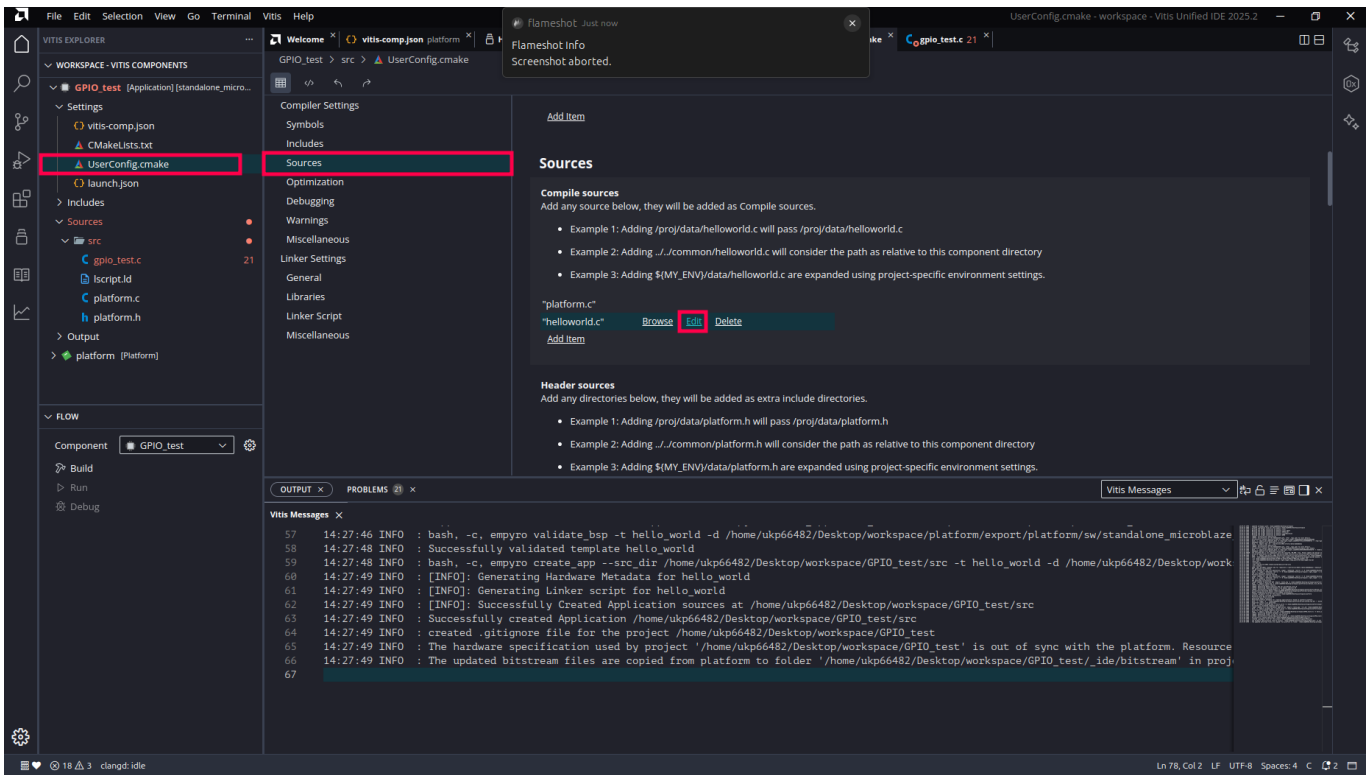
Add your source files under the `src` directory (e.g. `gpio_init.c`). Assembly files (`.s`) are also supported — add them the same way and they will be compiled and linked together with the rest of the project. See `workspace-example/examples/02_btn_led_asm/` for an example written entirely in RISC-V assembly.



4.3 Configure Compile Sources

Open **UserConfig.cmake** and go to the **Sources > Compile sources** section to add the new source files to the build:

1. Click **Browse** to select files
2. Verify the file appears in the list
3. Use **Delete** to remove unwanted entries

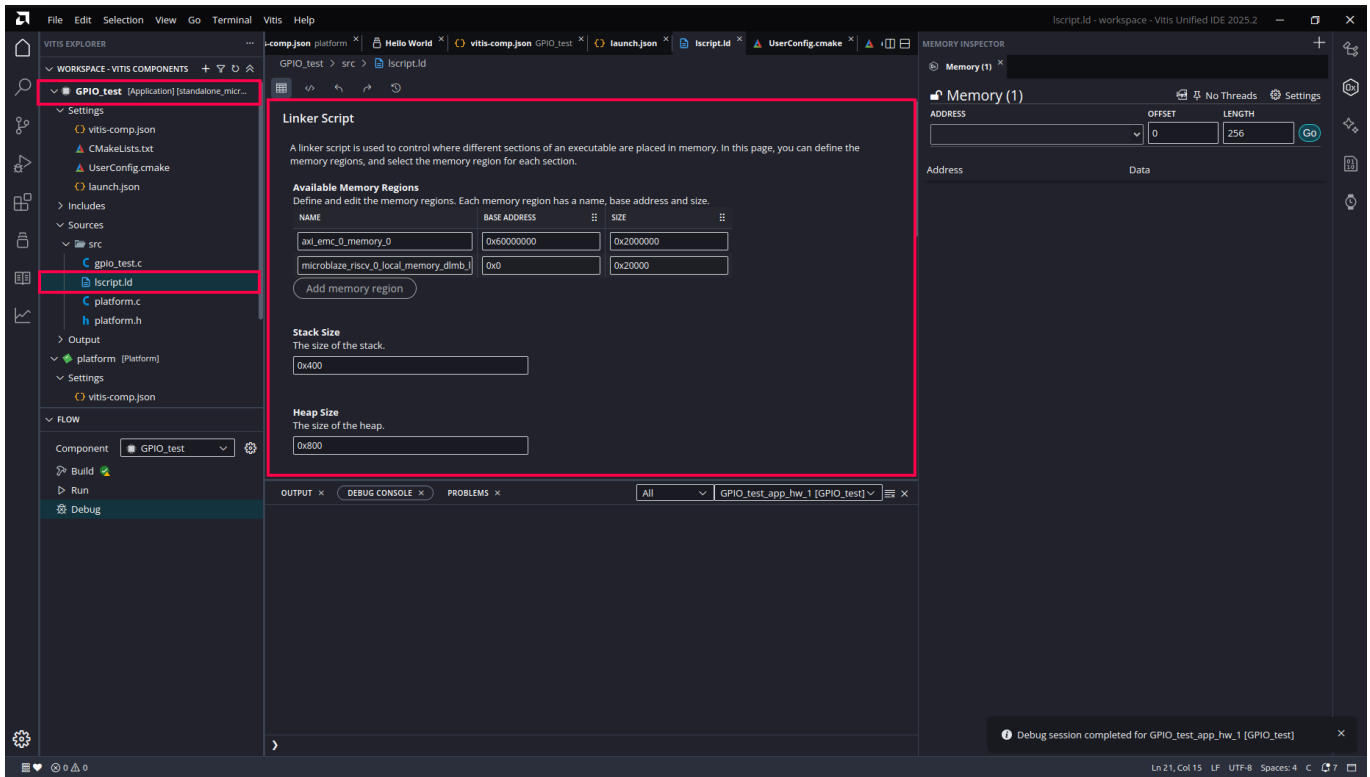


5. Configure the Linker Script

5.1 Memory Region Settings

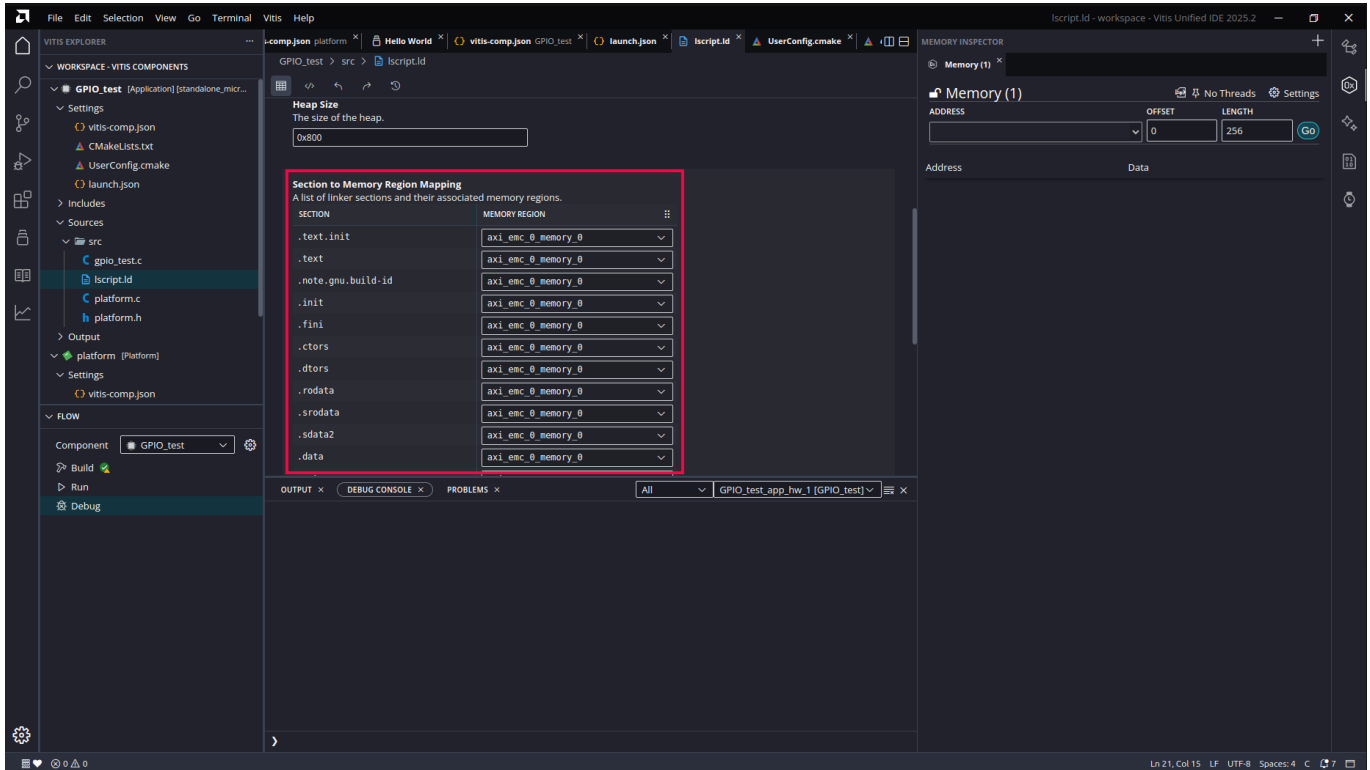
Open the **Linker Script** settings page to adjust:

- **Available Memory Regions:** view available regions (e.g. Local Memory)
- **Stack Size:** set the stack size
- **Heap Size:** set the heap size



5.2 Section to Memory Region Mapping

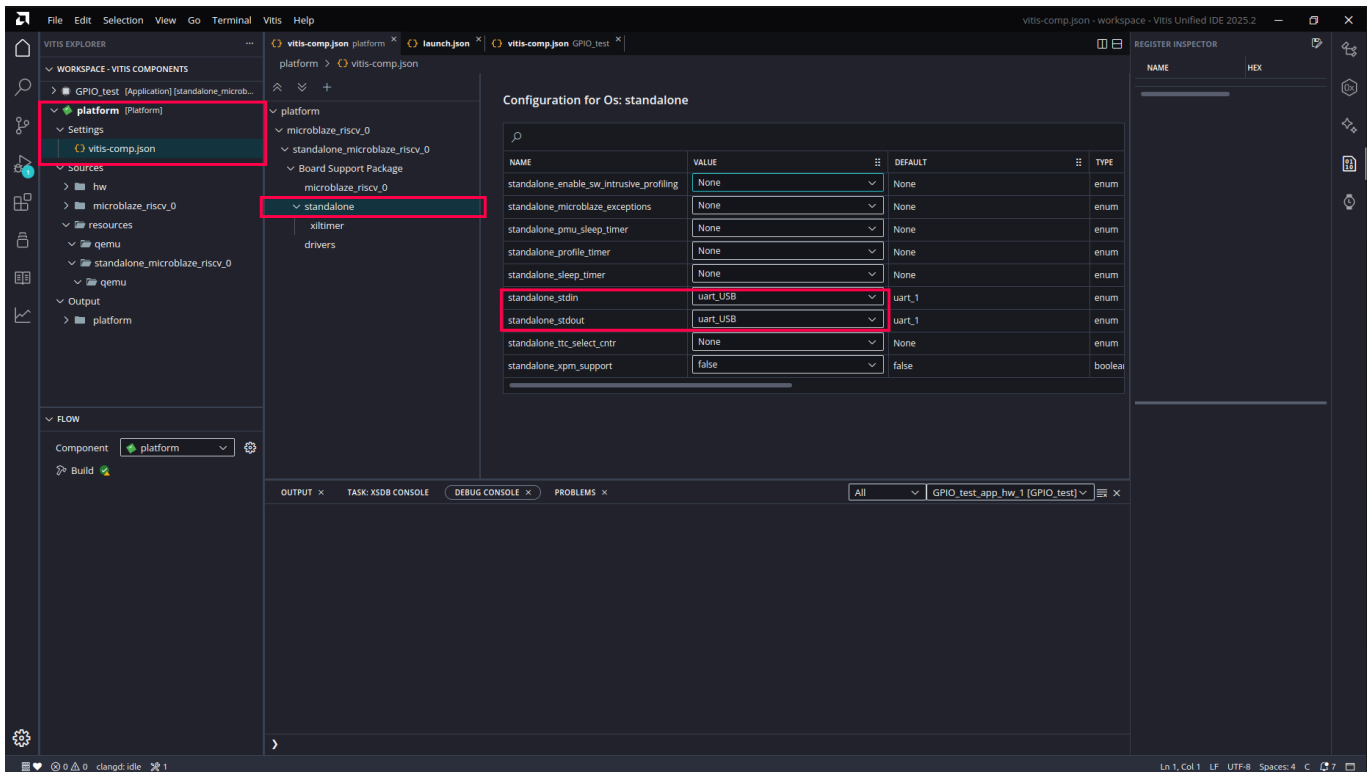
Verify that each section (`.text` , `.data` , `.bss` , `.stack` , `.heap` , etc.) is mapped to the correct memory region.



6. Configure Platform BSP

Open the platform's **Configuration for Os: standalone** settings page and verify the following key parameters:

- **stdin** and **stdout**: set to the appropriate UART peripheral (e.g. `axi_uartlite_0`)
- Adjust other BSP settings as needed

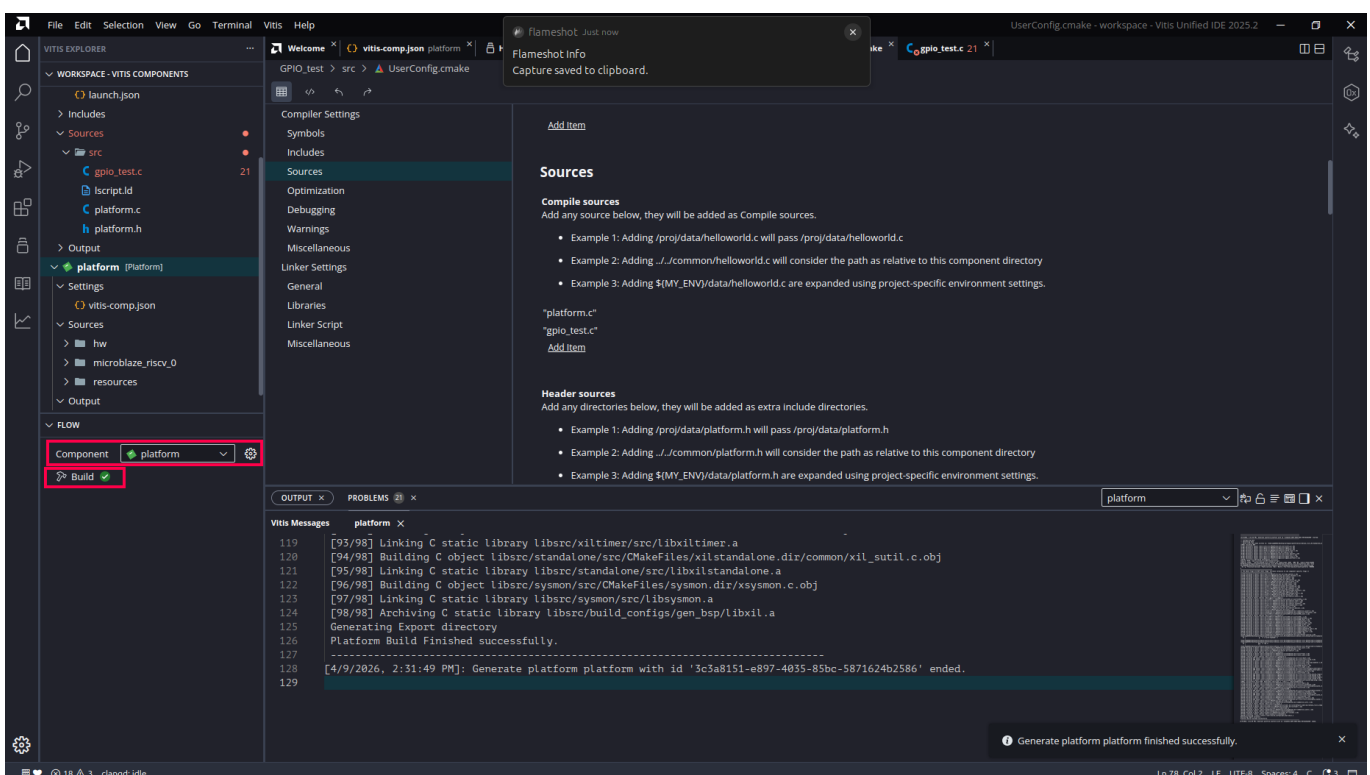


7. Build

7.1 Build the Platform

In the **FLOW** panel at the bottom-left, select the **platform** component and click **Build**.

Wait for the build to complete — confirm the log shows `Platform Build Finished Successfully`.

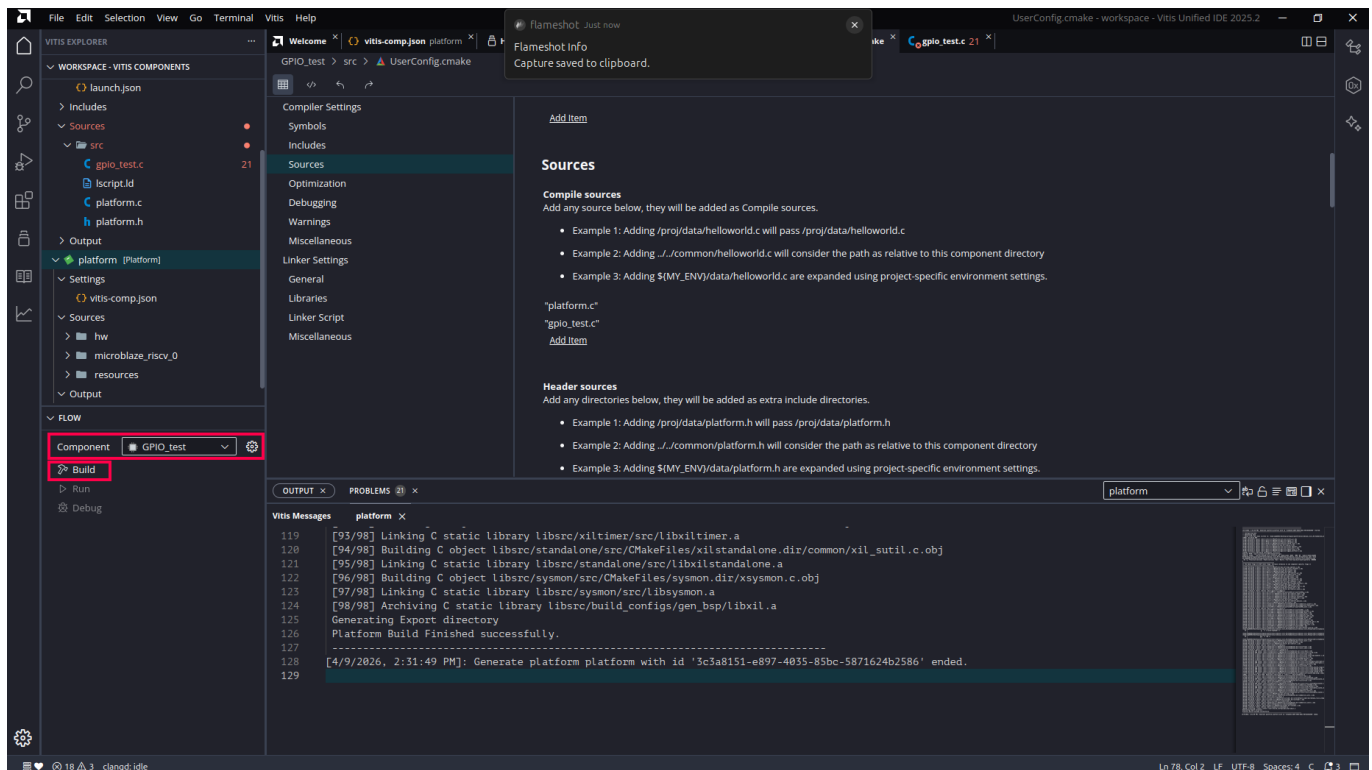


7.2 Build the Application

Switch to the application component (e.g. `GPIO_test`) and click **Build**.

Verify the build succeeds and produces the `.elf` file.

Note: During the build, Vitis automatically links in `boot.S` and `crt.o` from the standalone BSP. You do not need to provide them manually. - `boot.S` — the first code that runs after reset. It zeros all general-purpose registers (x0–x31), initializes the stack pointer, sets up exception/interrupt vectors, and jumps to the C runtime entry point. - `crt.o` (C Runtime) — runs before `main()` . It zeros the BSS segment (uninitialized global variables) and then calls `main()` .



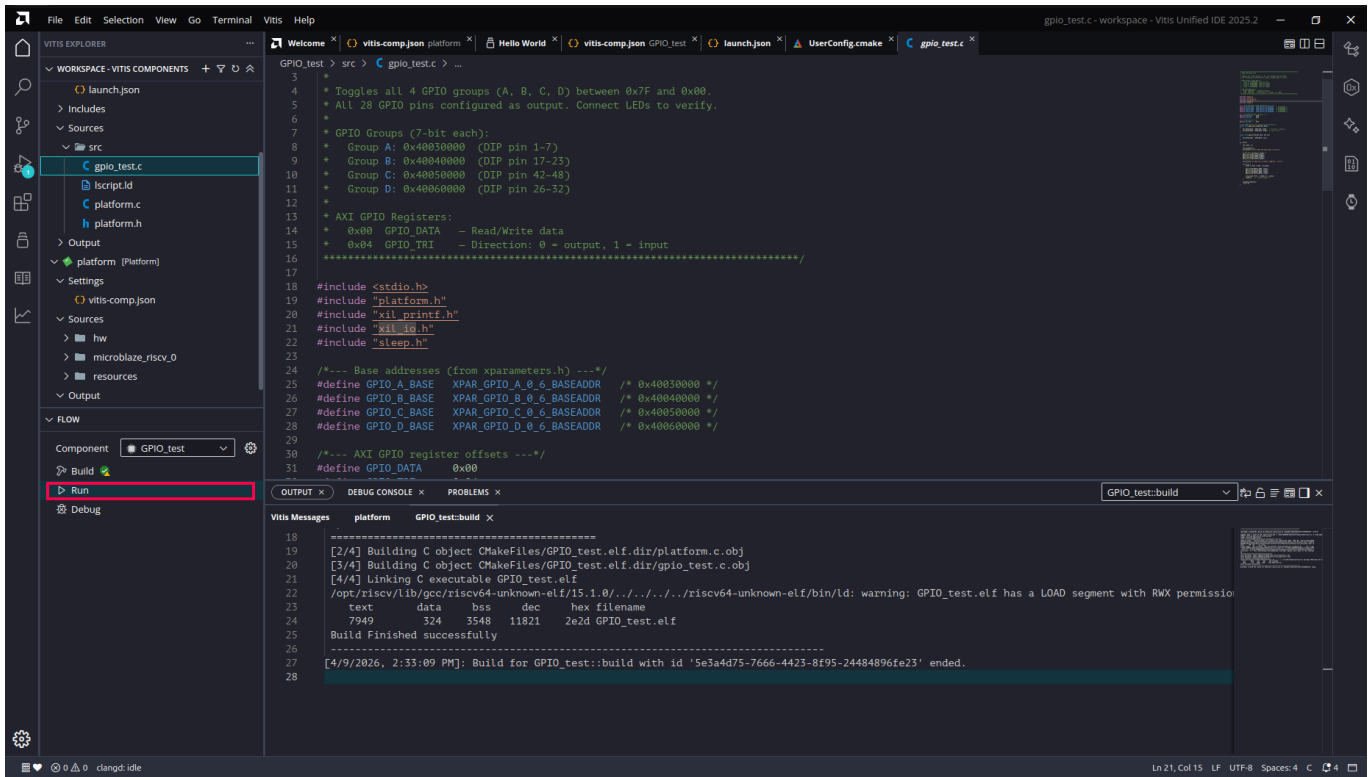
8. Run and Debug over JTAG

8.1 Run the Application

Before debugging, you can first run the application to verify it works correctly. In the **FLOW** panel, click **Run**.

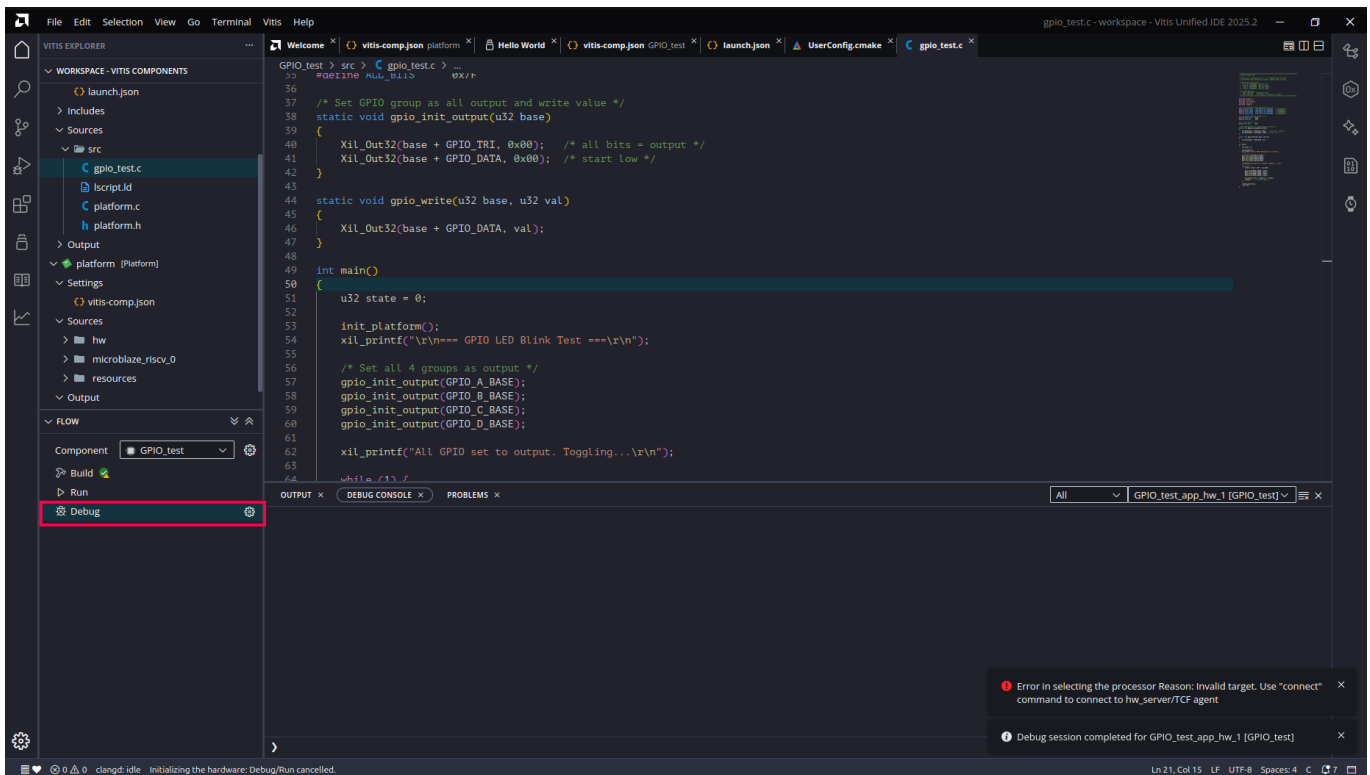
Vitis will automatically: 1. Download the bitstream to the FPGA via JTAG 2. Load the `.elf` into the MicroBlaze RISC-V processor 3. Execute the program from start to finish

This lets you confirm the application runs as expected before entering a debug session.



8.2 Start a Debug Session

Once the application is verified, click **Debug** in the **FLOW** panel to launch a debug session. This works similarly to GDB — the program is loaded and paused at `main()`, allowing you to inspect and step through the code interactively.

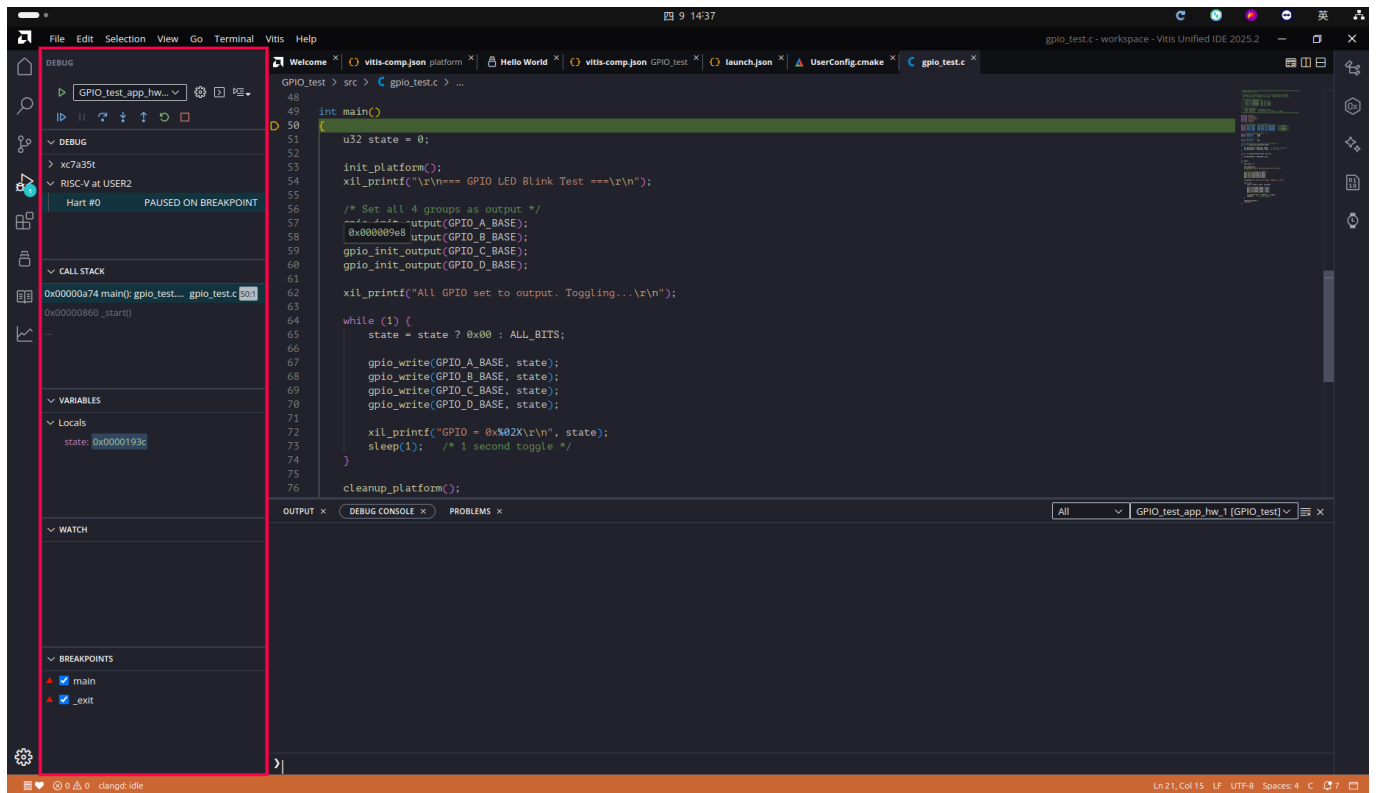


8.3 Debug Controls

In the Debug view, the left panel provides:

- **THREADS:** thread information
- **CALL STACK:** call stack trace
- **VARIABLES:** variable inspection
- **WATCH:** watch expressions
- **BREAKPOINTS:** breakpoint management

The toolbar at the top provides **Continue**, **Step Over**, **Step Into**, **Step Out**, and other debug controls.



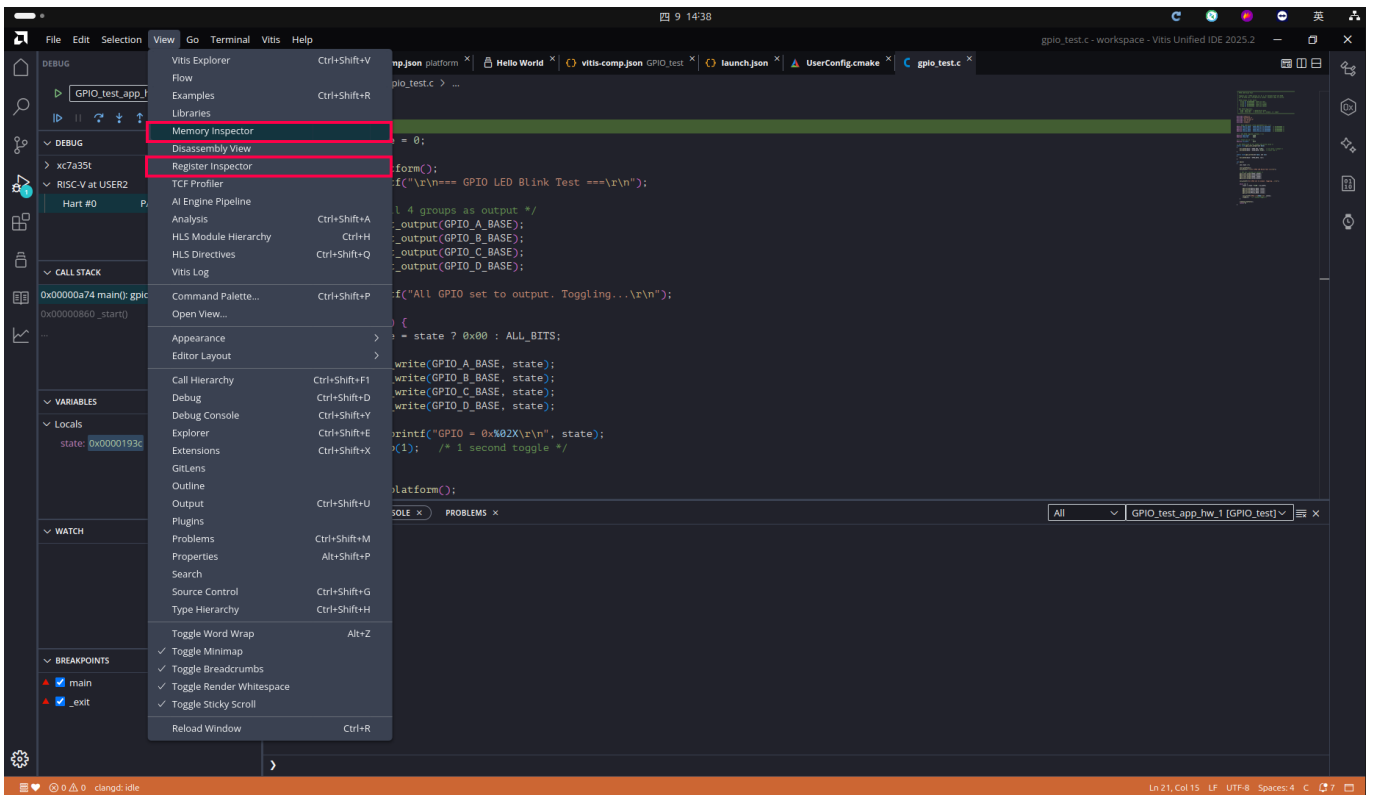
8.4 Breakpoints and Stepping

Click in the gutter area next to a line number to set a breakpoint. Use Step Over / Step Into to step through the code line by line.

9. Advanced Inspection Tools

9.1 Open the Register Inspector

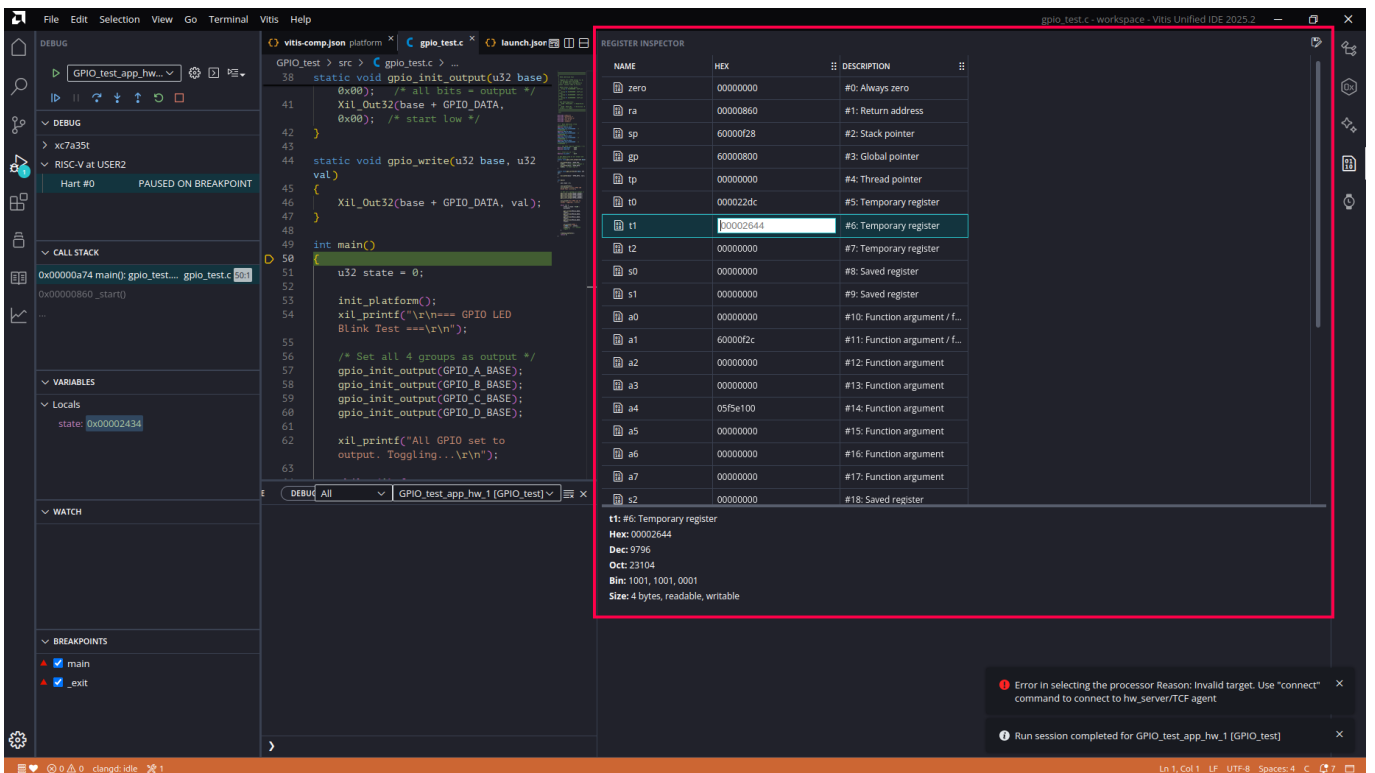
From the menu, select **View > Register Inspector** to open the register inspection panel.



9.2 View Register Contents

The Register Inspector displays all processor registers and their current values, including:

- General-purpose registers (x0 – x31)
- Program Counter (PC)
- HEX values and descriptions for each register



9.3 View Memory Contents

Open the **Memory Inspector** panel to:

- Enter a memory address (e.g. `0x60000000`) to inspect a specific memory region
- View data in hexadecimal format
- Monitor memory changes in real time

